

LINSI-OS

Diseño y fundamentos de una distribución GNU/Linux propia

Documento Fundacional



Autor: Juan Ignacio Wilt

Carrera: Ingeniería en Sistemas de Información – UTN FRLP

Ámbito: Laboratorio LINSI

(Laboratorio de Innovaciones en Sistemas de Información)

Fecha: septiembre de 2026

Resumen

Este trabajo presenta a LINSI-OS, una distribución GNU/Linux que se está construyendo desde cero, siguiendo la metodología conocida como Linux From Scratch, para cubrir las necesidades propias del Laboratorio LINSI. En lugar de instalar y configurar una distribución ya existente, se optó por compilar cada componente del sistema a partir de su código fuente, lo que permite decidir con precisión qué entra al sistema final y con qué configuración. El proyecto no arrancó de una lista de programas para instalar, sino de una serie de principios de diseño definidos antes de escribir el primer script de compilación: **seguridad, privacidad, robustez, estética y vanguardia**. Estos cinco pilares funcionan como criterio de decisión en cada etapa del desarrollo, y son los que terminan explicando por qué el sistema se construye como se construye. En las secciones siguientes se explica de dónde surge la idea, en qué consiste cada pilar, cuáles son los objetivos del proyecto, cómo está organizado técnicamente su desarrollo y por qué se lo considera un aporte original dentro del ámbito del laboratorio.

1. Introducción

1.1. De dónde surge la idea

El Laboratorio LINSI trabaja con equipos de hardware heterogéneo y se usan tanto para tareas de docencia como para investigación y desarrollo de software. Cualquier distribución de uso general resuelve ese tipo de diversidad con un compromiso pensado para una base de usuarios amplia, que no necesariamente coincide con lo que hace falta en un laboratorio universitario. A eso se suma que buena parte de las distribuciones orientadas a seguridad terminan siendo poco cómodas para el uso diario, o al revés: las distribuciones cómodas para el uso diario no ponen la seguridad como prioridad de diseño sino como un agregado posterior.

De esa tensión surgió la idea de LINSI-OS: en vez de partir de una distribución existente e ir ajustándola a fuerza de parches y configuraciones, construir un sistema propio en el que la seguridad, la privacidad y el resto de los principios que se explican en la Sección 2 estén presentes desde la primera decisión de diseño y no se agreguen después. La idea se armó antes de tocar una terminal: primero se definieron los

principios que el sistema tenía que sostener, y recién después se buscó, para cada uno de ellos, qué herramienta y qué configuración concreta lo hacía posible. Esa relación entre principio y herramienta es la que se documenta en este trabajo.

1.2. Por qué construir el sistema desde el código fuente

La alternativa más simple hubiera sido partir de una distribución ya armada (Debian, Arch Linux, o alguna orientada a seguridad como Kali o Qubes) e ir modificándola hasta acercarla a lo que se necesitaba. Se descartó ese camino por una razón concreta: al instalar paquetes binarios ya compilados por terceros, el control real sobre qué contiene cada componente del sistema es limitado. No se sabe con certeza qué opciones de compilación se usaron, qué dependencias quedaron incluidas sin necesidad, ni qué superficie de ataque adicional se está heredando de decisiones tomadas por otro equipo, con otros objetivos.

Siguiendo el enfoque de Linux From Scratch, en cambio, cada paquete desde el compilador cruzado hasta el entorno gráfico, se descarga en su forma de código fuente, se verifica contra su suma de verificación criptográfica oficial y se compila específicamente para el sistema de destino, sin heredar nada del sistema operativo que se usa para compilar. El costo de este enfoque es un esfuerzo de ingeniería mucho mayor que el de simplemente instalar paquetes: hay que resolver a mano cada dependencia, cada opción de configuración y cada incompatibilidad entre versiones. A cambio, se obtiene control total sobre el resultado final, y una comprensión real de qué es lo que efectivamente compone al sistema, que es, en definitiva, uno de los objetivos formativos del proyecto dentro del laboratorio.

2. Los cinco pilares del proyecto

Los cinco pilares que se describen a continuación (seguridad, privacidad, robustez, estética y vanguardia) no se pensaron como una lista de características deseables, sino como los criterios contra los que se mide cada decisión técnica del proyecto. Cuando en algún punto del desarrollo hay que elegir entre dos herramientas que cumplen la misma función, la pregunta que decide no es cuál es más popular o más fácil de configurar, sino cuál sostiene mejor alguno de estos cinco principios. Por eso cada pilar se presenta junto con ejemplos concretos de decisiones técnicas que lo

materializan; la lista completa de fases del desarrollo, con el detalle técnico de cada una, se retoma en la Sección 5.

2.1. Seguridad

Es el pilar central del proyecto, y el que más peso tuvo a la hora de descartar alternativas ya armadas. La idea es que el sistema resista tanto ataques lógicos como físicos, algo particularmente relevante en un laboratorio donde los equipos no siempre están bajo supervisión directa. A nivel de núcleo, esto se traduce en activar Kernel Lockdown para impedir que se modifique código en memoria incluso con privilegios de root, sumar KASLR para dificultar la explotación de vulnerabilidades mediante la aleatorización de direcciones, y deshabilitar puertos con acceso directo a memoria (DMA) que habitualmente quedan expuestos sin necesidad. A nivel de espacio de usuario, se reemplazan las utilidades clásicas escritas en C por una reimplementación en Rust, lo que elimina de raíz una familia entera de errores de manejo de memoria que durante décadas fue una de las causas más comunes de vulnerabilidades en sistemas Unix. El control de acceso obligatorio se resuelve con SELinux, que permite confinar un proceso comprometido a un conjunto mínimo de acciones autorizadas aunque ese proceso corra con privilegios elevados, y se complementa con un firewall de denegación por defecto y un registro de auditoría inmutable. Por último, la cadena de distribución de software también queda cerrada: el gestor de paquetes exige una firma criptográfica válida antes de instalar cualquier cosa, de modo que no haya forma de introducir una actualización maliciosa sin la clave privada del laboratorio.

2.2. Privacidad

Mientras que el pilar de seguridad apunta a que el sistema resista ataques, el de privacidad apunta a que ni el equipo ni el usuario queden expuestos frente a terceros durante el uso normal del sistema. Los datos en reposo se protegen cifrando el disco completo con LUKS2, usando Argon2id como función de derivación de clave; a diferencia de funciones más antiguas, Argon2id satura deliberadamente el uso de memoria durante el cálculo, lo que encarece de forma significativa un ataque de fuerza bruta acelerado por GPU. En red, la dirección MAC de las interfaces se aleatoriza en cada reinicio para dificultar el rastreo del equipo, y todas las resoluciones de nombres de dominio se cifran mediante DNS-over-TLS, de modo que ni el

proveedor de internet pueda ver qué sitios se visitan. La idea, en conjunto, es que la privacidad no dependa de que el usuario recuerde instalar y configurar herramientas adicionales, sino que sea el comportamiento por defecto del sistema.

2.3. Robustez

Un sistema pensado para uso diario en un laboratorio tiene que tolerar errores sin volverse inutilizable. Este pilar tiene dos frentes. Por un lado, la infraestructura del propio proyecto: la compilación se realiza en contenedores aislados y reproducibles, y la distribución de paquetes está pensada para correr sobre infraestructura propia del laboratorio en lugar de depender de un tercero. Por otro lado, la capacidad de recuperación del sistema instalado: se integra soporte para BTRFS en el núcleo, lo que permite tomar snapshots del disco en milisegundos, de forma que si una actualización o una configuración rompen algo, se puede volver a un estado funcional sin reinstalar nada. A esto se suma una elección deliberadamente conservadora en la biblioteca C del sistema: se usa glibc en lugar de alternativas más puristas en materia de seguridad, como musl, precisamente porque glibc garantiza compatibilidad con la mayor cantidad posible de software ya existente, y eso importa cuando el sistema se usa todos los días para tareas concretas del laboratorio y no solamente como banco de pruebas.

2.4. Estética

Este pilar responde a una crítica bastante habitual hacia las distribuciones orientadas a seguridad: suelen tener interfaces poco cuidadas, pensadas para especialistas y poco amigables para el uso diario. La apuesta de LINSI-OS es que un sistema endurecido no tiene por qué sentirse hostil. Eso se traduce en decisiones concretas: una terminal Zsh con información contextual, colores y autocompletado en lugar de la Bash por defecto, un escritorio KDE Plasma 6 con un tema oscuro unificado y tipografías con soporte de iconos para programación (Nerd Fonts) configuradas de fábrica para cualquier usuario nuevo, y un instalador gráfico personalizado con la identidad visual del laboratorio, de modo que el primer contacto con el sistema (la instalación misma) ya transmita que se trata de un producto cuidado y no de un ensamblado de herramientas sueltas.

2.5. Vanguardia

El último pilar tiene que ver con no cargar con decisiones técnicas heredadas simplemente porque en algún momento fueron el estándar. El caso más claro es el del servidor gráfico: en lugar de mantener compatibilidad con X11, que arrastra fallas de diseño conocidas desde hace años (como la posibilidad de que cualquier ventana capture las pulsaciones de teclado de otra), LINSI-OS se construye exclusivamente sobre Wayland. Lo mismo pasa con el audio, donde PipeWire reemplaza por completo a PulseAudio, y con la autenticación, donde se integra PAM con soporte nativo para tokens físicos FIDO2, el estándar actual en entornos corporativos para la verificación en dos pasos. Por último, el modelo de aplicaciones de terceros se apoya en Flatpak y en entornos aislados mediante Podman y Distrobox, lo que permite mantener el núcleo del sistema limpio y a la vez dar acceso a software comercial o a entornos de desarrollo específicos sin comprometer ni la seguridad de la base ni el rendimiento de la GPU.

3. Objetivos

3.1. Objetivo general

Diseñar, construir y documentar una distribución GNU/Linux propia, endurecida en materia de seguridad, apta para el uso cotidiano del Laboratorio LINSI en tareas académicas, de investigación y de desarrollo de software.

3.2. Objetivos específicos

- Construir un entorno de compilación cruzada completamente aislado del sistema anfitrión, de forma que cualquier integrante del laboratorio pueda reproducir exactamente el mismo entorno de build.
- Compilar un núcleo Linux endurecido, con mitigaciones activas contra las clases de ataque más relevantes: manipulación de memoria del núcleo, ataques por acceso directo a memoria y arranque no verificado.
- Construir un espacio de usuario completo y coherente sobre ese núcleo: biblioteca C, gestor de servicios y utilidades básicas.

- Incorporar una estrategia de seguridad en profundidad, que combine cifrado de disco, control de acceso obligatorio, un firewall de denegación por defecto y auditoría inmutable.
- Asegurar las comunicaciones de red y la autenticación de usuarios, incluyendo autenticación de dos factores con tokens físicos FIDO2.
- Diseñar una infraestructura propia de distribución de paquetes, firmados criptográficamente, con integración continua sobre infraestructura del laboratorio.
- Construir un entorno gráfico moderno basado en Wayland, con aceleración de hardware real para las tres familias de GPU presentes en el laboratorio, y el entorno de escritorio KDE Plasma 6.
- Producir, en una etapa posterior, una imagen instalable mediante un instalador gráfico propio.

4. Cómo está construido el sistema

4.1. Entorno de compilación aislado

Todo el proceso de compilación corre dentro de un contenedor Podman, orquestado con podman-compose, que no comparte bibliotecas ni herramientas con el sistema del desarrollador. Esta decisión cumple dos funciones: evita que algo instalado en la máquina de quien compila termine filtrándose sin querer al sistema final, y garantiza que cualquier persona del laboratorio pueda levantar exactamente el mismo entorno de build en su propia computadora, sin depender de configuraciones particulares de cada máquina. El proyecto se planteó originalmente sobre Docker Desktop, pero se migró a Podman al encontrar problemas de funcionamiento en el entorno disponible.

4.2. Compilación cruzada

El sistema se compila para un triplete propio, x86_64-linsi-linux-gnu, mediante un compilador cruzado construido en la primera etapa del desarrollo. A diferencia de un proceso de arranque (bootstrap) clásico de Linux From Scratch, el proyecto no hace chroot al sistema de archivos de destino durante la construcción: cada paquete se instala usando la combinación estándar de Autotools, Meson o CMake con --

prefix=/usr y la variable DESTDIR apuntando al sistema de archivos final, lo que permite compilar sin necesidad de ejecutar directamente binarios de destino dentro del entorno de build. Esto tiene una consecuencia técnica importante: un binario recién compilado para el sistema de destino no siempre puede ejecutarse dentro del propio contenedor de build, aunque comparta la misma arquitectura de conjunto de instrucciones, porque sus dependencias dinámicas fueron compiladas para ese sistema final y no para el contenedor. Resolver esa clase de situaciones (identificar cuándo hace falta una herramienta específicamente nativa en lugar de la versión cruzada) fue una de las partes más delicadas del desarrollo y forma parte del trabajo de ingeniería documentado internamente en el proyecto.

4.3. Organización del trabajo en scripts independientes

Cada etapa del desarrollo se implementa como un script de shell autocontenido, que no depende de otros scripts de etapas anteriores y que define sus propias funciones de registro, verificación y manejo de errores. Cada script expone pasos individuales que se pueden ejecutar por separado, además de un modo que corre la etapa completa de punta a punta. Cada paso relevante termina con una verificación automática que confirma, inspeccionando los archivos generados, que el resultado es el esperado antes de pasar al siguiente paso. Todas las ejecuciones quedan registradas en archivos de log con marca temporal, lo cual resultó fundamental para poder diagnosticar errores de compilación bastante específicos, sobre todo en las etapas más complejas del proyecto, relacionadas con la compilación cruzada de LLVM y de la pila gráfica.

4.4. Control de versiones

El desarrollo se lleva bajo control de versiones con Git, con un registro de cambios que documenta cada corrección técnica relevante como un commit individual. Esto permite mantener un historial útil ante la eventual necesidad de revertir un cambio, y deja trazabilidad de cada decisión tomada durante el desarrollo, algo que también resulta valioso a la hora de sostener, con evidencia concreta, que se trata de un trabajo de autoría propia desarrollado a lo largo del tiempo y no de un proyecto armado de una sola vez.

5. Organización del desarrollo en fases

El desarrollo técnico se organiza en ocho fases incrementales, donde cada una depende de los artefactos producidos por la anterior y es responsable de una capa concreta del sistema. Esta organización en capas responde a la misma lógica que se explicó en la sección anterior: minimizar el acoplamiento entre etapas, de forma que cada una se pueda desarrollar, probar y depurar de manera aislada. A grandes rasgos, las ocho fases se agrupan en cuatro bloques temáticos.

5.1. Cimientos y sistema base

El primer bloque cubre la Fase 0, donde se construye el compilador cruzado completo (Binutils y GCC) junto con las cabeceras del núcleo Linux necesarias para forjar binarios completamente independientes del sistema anfitrión; la Fase 1, donde se compila el núcleo Linux endurecido descrito en el pilar de seguridad; y la Fase 2, donde se construye el espacio de usuario completo: la biblioteca C, systemd como gestor de servicios, las utilidades base reescritas en Rust y Zsh como intérprete de línea de comandos por defecto.

5.2. Seguridad, red y repositorios

El segundo bloque agrupa la Fase 3, donde se implementa la estrategia de seguridad en profundidad (cifrado de disco, SELinux, firewall y auditoría); la Fase 4, donde se asegura la capa de comunicaciones (red, DNS cifrado y autenticación con FIDO2); y la Fase 5, donde se diseña la infraestructura autónoma de distribución de paquetes firmados, pensada para correr sobre un servidor propio del laboratorio detrás de un túnel de acceso seguro, con integración continua que compile y firme automáticamente cada nueva versión de un paquete.

5.3. Entorno gráfico y estética

El tercer bloque corresponde a la Fase 6, la más extensa del proyecto, donde se construye todo el entorno gráfico: el protocolo Wayland y su ecosistema de dependencias (entrada de dispositivos, audio moderno con PipeWire, renderizado de fuentes), la biblioteca gráfica Mesa junto con LLVM y Clang (este último necesario para compilar internamente los shaders que usan los drivers gráficos de Intel) para

dar soporte de aceleración por hardware a las tres familias de GPU del laboratorio, y por último el framework Qt6 y el entorno de escritorio KDE Plasma 6 sobre el que se aplican las decisiones de estética descritas en la Sección 2.

5.4. Ecosistema híbrido y despliegue

El último bloque contempla la Fase 7, donde se incorpora Flatpak para instalar software de terceros sin tocar la base del sistema, y Podman o Distrobox para aislar entornos de desarrollo y pruebas académicas en contenedores efímeros; y la Fase 8, donde el sistema se empaqueta en una imagen SquashFS de solo lectura y se distribuye mediante un instalador gráfico propio, basado en Calamares, personalizado con la identidad visual del laboratorio.

Esta división en cuatro bloques y ocho fases no es solamente organizativa: es también la traducción directa de los cinco pilares descritos en la Sección 2 a un plan de trabajo concreto. Cada fase existe porque sostiene a uno o más pilares, y ninguna herramienta entra al sistema si no puede justificarse contra alguno de esos cinco principios.

6. Originalidad del proyecto

LINSI-OS no es una configuración particular de una distribución existente ni un conjunto de scripts para automatizar la instalación de paquetes ya armados por terceros. Es un diseño propio que arranca de cinco principios definidos antes de cualquier implementación técnica, y que se traduce en una arquitectura de construcción desde el código fuente en la que cada componente del sistema final es una elección justificada y no una herencia del sistema operativo usado para compilar. Cabe aclarar que ninguna de las decisiones tomadas individualmente es, por sí sola, exclusiva de este proyecto: la migración hacia utilidades base reescritas en Rust es una tendencia que ya adoptaron distribuciones mayores (Ubuntu la incorpora como opción por defecto desde su serie 25.10/26.04 LTS), y el abandono de X11 en favor de un entorno exclusivamente Wayland también es un camino que distribuciones como Fedora y la propia Ubuntu ya empezaron a recorrer en sus ediciones GNOME. Lo que sí resulta distintivo, hasta donde llegó una búsqueda de antecedentes realizada en septiembre de 2026, es la combinación particular de estas decisiones

sobre una base construida enteramente desde cero: integrar una reimplementación en Rust de las utilidades base, exigir firma criptográfica obligatoria para cualquier paquete sobre infraestructura de distribución propia del laboratorio, incorporar autenticación FIDO2 de forma nativa, y construir el entorno gráfico exclusivamente sobre Wayland y PipeWire sin arrastrar compatibilidad con X11 ni PulseAudio; todo ello dentro de un entorno de compilación cruzada completamente aislado del sistema anfitrión, en lugar de partir de una distribución ya existente y reconfigurarla. El antecedente más cercano identificado, secureblue (una distribución endurecida construida sobre la base de Fedora Atomic), comparte el espíritu de priorizar la seguridad desde el diseño, pero no reúne el resto de estas decisiones: no se compila desde una base propia aislada del sistema anfitrión, no reemplaza las utilidades base por una reimplementación en Rust, no exige autenticación FIDO2 de forma nativa y no distribuye sus paquetes mediante infraestructura de firma propia. Esta afirmación se sostiene sobre la evidencia relevada a la fecha, no sobre una revisión exhaustiva de antecedentes, por lo que se formula como una observación fundamentada y no como una negación categórica. A esto se suma que el desarrollo completo queda documentado paso a paso: cada script de compilación, cada corrección técnica y cada decisión de diseño quedan registrados bajo control de versiones, con fecha y con la justificación concreta que llevó a tomarla. Ese registro es, en sí mismo, una parte central del aporte del proyecto: no se trata solamente de llegar a un sistema operativo funcional, sino de dejar documentado con precisión cómo se construyó, con qué criterios y resolviendo qué problemas concretos, de forma que el trabajo pueda sostenerse, revisarse y continuarse por cualquier otro integrante del laboratorio.

7. Conclusión

LINSI-OS plantea una alternativa concreta frente a la disyuntiva habitual entre usar una distribución genérica o adaptarla a fuerza de configuraciones parciales: construir, desde el código fuente y con una estrategia de seguridad en profundidad, un sistema operativo pensado específicamente para las necesidades del Laboratorio LINSI. La decisión de definir primero los principios de diseño (seguridad, privacidad, robustez, estética y vanguardia) y recién después derivar de ellos la arquitectura técnica, es lo que le da coherencia al proyecto de punta a punta: cada herramienta que forma parte

del sistema puede explicarse en función de alguno de esos cinco pilares, y no como una elección aislada o de conveniencia.

El presente documento busca dejar constancia, ante el Laboratorio LINSI y ante quien corresponda evaluar el reconocimiento de la autoría de la idea, de en qué consiste el proyecto, de dónde surge y por qué se lo considera un desarrollo original: no por usar tecnología novedosa aisladamente, sino por la forma particular en que esas tecnologías se combinan alrededor de un criterio de diseño definido de antemano y sostenido de manera consistente a lo largo de todo el desarrollo.